

Шпаргалка по Docker

Контейнеры и образы, сборка, тома и сети, Compose, диагностика и уборка мусора. Вместо значений в угловых скобках подставьте свои.

Запуск контейнеров

```
docker run <образ>
    Создать и запустить контейнер; образ при
    необходимости скачается

docker run -d --name web -p 8080:80 nginx
    В фоне, с именем, порт 8080 хоста на 80 внутри
    контейнера

docker run -it --rm ubuntu bash
    Интерактивно с терминалом; после выхода контейнер
    удаляется

docker run -e KEY=value -v $PWD:/app -w /app node:20
npm test
    Переменная окружения, текущий каталог внутрь
    контейнера, рабочий каталог и команда

docker run --restart unless-stopped ...
    Перезапускать после сбоя и после перезагрузки хоста
```

Жизненный цикл

```
docker ps / docker ps -a
    Запущенные контейнеры; с ключом — все, включая
    остановленные

docker stop <имя> / docker start <имя>
    Остановить (сигнал SIGTERM, через 10 с — SIGKILL) и
    запустить снова

docker restart <имя>
    Перезапуск

docker rm <имя>
    Удалить остановленный контейнер (-f —
    принудительно)

docker rename <старое> <новое>
    Переименовать
```

Заглянуть внутрь

```
docker logs -f --tail 100 <имя>
    Журнал контейнера в реальном времени, последние
    100 строк

docker exec -it <имя> sh
    Открыть оболочку в работающем контейнере

docker inspect <имя>
    Полные сведения в JSON: сети, тома, переменные,
    состояние

docker stats
    Потребление процессора, памяти и сети в реальном
    времени

docker top <имя>
    Процессы внутри контейнера

docker cp <имя>:/путь ./локальный
    Скопировать файлы из контейнера и обратно
```

Образы

```
docker images
    Список локальных образов

docker pull <образ>:<тег>
    Скачать образ из реестра

docker build -t myapp:1.0 .
    Собрать образ из Dockerfile в текущем каталоге

docker build --no-cache --build-arg VER=1.2 .
    Сборка без кеша слоёв и с передачей аргумента

docker tag myapp:1.0 registry.example.com/myapp:1.0
    Добавить имя для публикации в реестр

docker push registry.example.com/myapp:1.0
    Отправить образ в реестр (перед этим docker login)

docker history <образ>
    Слои образа и их размер — видно, что раздувает
    сборку

docker save / docker load
    Выгрузить образ в архив и загрузить на другой машине
```

Dockerfile: основные инструкции

```
FROM node:20-alpine
    Базовый образ; несколько FROM — многоэтапная
    сборка

WORKDIR /app
    Рабочий каталог для последующих инструкций

COPY package*.json ./
    Копирование файлов; сначала зависимости — тогда
    кеш слоёв срабатывает чаще

RUN npm ci --omit=dev
    Команда на этапе сборки; каждая создаёт слой

ENV NODE_ENV=production
    Переменная окружения в итоговом образе

EXPOSE 3000
    Документирует порт; сам по себе ничего не публикует

USER node
    Не запускать приложение от root

HEALTHCHECK CMD wget -qO- localhost:3000/health || exit
1
    Проверка работоспособности контейнера

ENTRYPOINT ["node"] / CMD ["server.js"]
    Неизменяемая часть команды и аргументы по
    умолчанию
```

```
# Многоэтапная сборка
FROM golang:1.22 AS build
WORKDIR /src
COPY . .
RUN go build -o /app ./cmd/server

FROM gcr.io/distroless/base
COPY --from=build /app /app
CMD ["/app"]
```

Томы и данные

```
docker volume create data
    Создать именованный том — предпочтительный способ
    хранить состояние

docker run -v data:/var/lib/postgresql/data postgres:16
    Подключить том в контейнер

docker run -v $PWD/src:/app/src:ro ...
    Каталог хоста внутрь контейнера, только для чтения

docker volume ls / inspect / rm
    Список, сведения, удаление
```

Осторожно: данные в файловой системе контейнера исчезают вместе с ним. Всё, что нужно сохранить, держите в томах.

Сети

```
docker network create app-net
    Своя сеть; контейнеры в ней видят друг друга по
    именам

docker run --network app-net --name db postgres:16
    Подключение при запуске: соседи обращаются к нему
    по имени db

docker network connect app-net <контейнер>
    Подключить уже работающий контейнер

-р 127.0.0.1:5432:5432
    Опубликовать порт только на локальный интерфейс, а
    не наружу
```

Docker Compose

```
docker compose up -d
    Поднять описанные сервисы в фоне

docker compose up -d --build
    Пересобрать образы перед запуском

docker compose ps / logs -f [сервис]
    Состояние и журналы

docker compose exec <сервис> sh
    Оболочка внутри сервиса

docker compose down
    Остановить и удалить контейнеры и сети (-v — вместе
    с томами)

docker compose config
    Показать итоговый файл с подставленными
    переменными
```

```
services:
  web:
    build: .
    ports: ["8080:3000"]
    environment:
      DATABASE_URL: postgres://app@db/app
    depends_on: [db]
  db:
    image: postgres:16
    volumes: ["pgdata:/var/lib/postgresql/data"]
volumes:
  pgdata:
```

Уборка

```
docker system df
    Сколько места занимают образы, контейнеры, тома и
    кеш сборки

docker container prune
    Удалить все остановленные контейнеры

docker image prune -a
    Удалить образы, не используемые ни одним
    контейнером

docker builder prune
    Очистить кеш сборки — обычно он и занимает больше
    всего

docker system prune -a --volumes
    Полная уборка вместе с томами
```

Осторожно: `--volumes` удаляет данные, не привязанные к запущенным контейнерам. Проверьте `docker volume ls` заранее.

Когда что-то не работает

- Контейнер сразу завершился — `docker logs` и `docker inspect --format '{{.State.ExitCode}}'`.
- Порт занят — `ss -ltnp` на хосте; публикуйте другой порт.
- Приложение не видит соседа — контейнеры должны быть в одной сети, обращение по имени сервиса, а не по `localhost`.
- Права на файлы в томе — сверьте UID процесса в контейнере и владельца каталога на хосте.
- Образ слишком большой — `docker history`, многоэтапная сборка, `.dockerignore`.